

AZURE-DE (Azure, ADF, Synapse)

5. Azure	6. ADF	15. Synapse	16. Azure Devops
----------	--------	-------------	------------------

Azure:

1. Cloud Computing: [delivery of virtual compute services\(compute,storage,network...etc\) over Internet on pay as u use basis](#)
2. Cloud Providers: [Amazon\(AWS,2006\)](#), [Microsoft\(Azure,2010\)](#), [Google\(GCP,2008\)...etc](#)
3. Azure Introduction:
4. Create Azure Trial A/C :

Signup Azure Account through <https://azure.microsoft.com/cloud> --> Next login with portal.azure.com

5. Getting started with Azure: [Create RG, ADLS, SQL databases, ADF...etc and users in EntraID\(AzureAD\)](#)
6. Create datalake: creating ADLS

StorageAccount --> RG, Name, Region --> Create # first it will be Blob, when we enable Hierarchical Namespace it becomes ADLS
Next upload needed files in folders under containers

7. Create Azure SQL server & DB

Azure SQLDB --> create --> RG, DBName(nn-db), select Server(Name,Region,username,password,compute-Tier) --> Create
Inside Server dashboard --> Set server Firewall --> public access(selected network),Add client IPv4, tick allow azure
services,resources. Next go to databases(leftside panel) --> nn-db --> Query Editor --> login --> run queries

8. Create Data Factory: [RG, Name\(nnadf\), Region --> create --> Launch Studio --> Author --> create pipelines](#)
9. Upgrade to Pay as you go: Free --> Paid

Azure ADF:

1. Introduction :
2. [Copy data within ADLS :](#)

Create ADLS with input(OrdersNew.csv), output Folders.
Create ADF --> New pipeline --> drag Copy --> Source[dataset(ADLS, OrdersNew.csv file), linked service(ADLS)],
Sink[dataset(ADLS,csv,output), linked service(ADLS)] --> Debug, Publish

3. [Copy Entire Folder/Specific files :](#)

drag Copy --> Source[dataset(ADLS,csv,Input, wildcard Filepath(Input/*.csv)), linked service(ADLS)], Sink[dataset(ADLS,csv,output),
linked service(ADLS)] --> Debug, Publish
drag Copy --> Source[dataset(ADLS,csv,Input, wildcard Filepath(Input/Sales*.csv)), linked service(ADLS)], Sink[dataset(ADLS,csv,output),
linked service(ADLS)] --> Debug, Publish

4. [Copy data from ADLS --> Azure SQL DB :](#)

Create ADLS with input(OrdersNew.csv), output Folders and Create Azure SQL DB
drag Copy --> Source[dataset(ADLS,OrdersNew.csv), linked service(ADLS)], Sink[dataset(Azure-SQLDB,AutoCreateTable, Table(dbo.Orders)),
linked service(SQL)] --> Debug, Publish
drag Copy --> Source[dataset(SQL,Azure-SQLDB, select Orders Table), linked service(SQL)], Sink[dataset(ADLS,csv,output, Orders.csv),
linked service(ADLS)] --> Debug, Publish

5. [Copy data Additional Columns :](#)

Create ADLS, YearWise Folder with 2011.csv, 2012.csv, 2013.csv files
drag Copy --> Source[dataset(ADLS,wildcard Filepath(YearWise/*.csv),Addl Columns(FileName,\$\$FILENAME)), linked service(ADLS)],
Sink[dataset(Azure-SQLDB,AutoCreateTable, Table(dbo.tblOrders)), linked service(SQL)] --> Debug, Publish
Note: we can use our needed extra columns(FileName,FilePath,Timestamp...etc) from Source(Addl Columns dropdown list)

6. [Set VariableActivity :](#)

ADF --> pipeline --> Variables(+New) --> (x,(string/boolean/array/Int)), (y,Int)
drag SetVariable --> SetVariable1[Settings[Name(x),Value(India)]
drag SetVariable --> SetVariable2[Settings[Name(y),Value(5000)]]

7. [Copy Files in Time Frame \(Ex: Copy 1 hour back files\) :](#)

Create ADLS, YearWise Folder with 2011.csv, 2012.csv, 2013.csv files
drag Copy --> Source[dataset: ADLS,wildcard
Filepath(YearWise/*.csv),FilterbyLastModified(starttime(dynamic,@subtractFromTime(utcnow(),1,'Hour'))),(endtime(@utcnow()))],linked
service(ADLS)]
Sink[dataset(ADLS,csv,output),linked service(ADLS)] --> Debug, Publish

8. [Get MetadataActivity :](#)

Create ADLS, input Folder with some csv files
drag GetMetadata --> Settings[dataset: ADLS,csv,input folder, FiledList:(+New) -->
Argument(ChildItems/ColumnCount/Size/LastModified/Exists/ItemName...etc)] --> debug, publish

9. [ForEach Activity :](#)

ADF --> pipeline --> parameter(Name(Numbers),Array,[1,2,3,4]),parameter(Name(Countries),Array,['India','US','UK','Germany']),
Variable1(x,int), Variable2(y,string)
drag ForEach -->
Settings[Items(dynamic,@pipeline().parameters.Numbers)],Activities(SetVariable(Settings[Name(x),Value(dynamic,@item())])) --> debug,
publish
drag ForEach -->
Settings[Items(dynamic,@pipeline().parameters.Countries)],Activities(SetVariable(Settings[Name(y),Value(dynamic,@item())])) --> debug,
publish

10. [Copy Each File to New Table :](#)

Create ADLS, YearWise Folder with MyOrders.csv, MyPurchase.csv, MySales.csv files
ADF --> pipeline --> variable(FileName,string)
GetMetadata --> dataset: ADLS,csv,input, FiledList(ChildItems)
ForEach --> [Items(dynamic,@activity('Get files').output.childItems),
Activities[SetVariable(Name(FileName),Value(dynamic,@item.name)),
drag Copy[Source(dataset:ADLS,csv,input@dataset().SourceFileName),linkservice(ADLS),Parameters(SourceFileName,string),
SourceFileName(@variables('FileName'))],

```
Sink(dataset:SQLDB,linkservice(SQLDB),AutoCreateTable,dbo.@dataset().tablename),Parameters(tablename,string),tablename(@replace(variables('FileName'),''.csv',''))]] --> debug, publish
```

11. Truncate & Load :

```
drag Copy  
[Source(dataset:ADLS, csv, input/dailsales.csv, linkservice(ADLS)), Sink(dataset:SQLDB, linkservice(SQLDB), AutoCreateTable, dbo.tbldailySales)] --> debug, publish  
Next Sink[click None, uncheck Auto-Create Table option, precopy script(truncate table dbo.tbldailySales)] --> debug, publish
```

12. Copy data with Upsert :

```
# Copy data with Upsert (update+Insert) Primary Column should be present in input csv  
drag Copy  
[Source(dataset:ADLS, csv, input/dailsales.csv, linkservice(ADLS)), Sink(dataset:SQLDB, linkservice(SQLDB), AutoCreateTable, dbo.tbldailySales)] --> debug, publish  
Next Sink[Write behavior(Upsert), key columns(+New), select Date column] --> debug, publish
```

13. Append Variable :

```
ADF --> pipeline --> Variable(+New)-->(x, Array), (output, Array)  
drag AppendVariable --> Settings(x, 100)  
drag SetVariable --> Settings(output, dynamic(@variables('x')))  
drag AppendVariable --> Settings(x, 200)  
drag SetVariable --> Settings(output, dynamic(@variables('x')) # Append1-->Set1-->Append2-->Set2 --> final output ["100", "200"]
```

14. Copy data activity (Using SQL queries) :

```
Copy[Source(dataset: SQLDB, linkservice(SQLDB), select query, Query(Select * from SalesTran)), Sink(dataset:SQLDB, linkservice(SQLDB), AutoCreateTable, dbo.Salesdata)] --> debug, publish
```

15. Column Mapping in Copy activity :

```
Create ADLS with orders.csv file and SQL DB with Table(tblorders1) & schema match with csv file columns, without data  
drag copy[Source(dataset:ADLS, csv, input/orders.csv, linkservice(ADLS)), Sink(dataset:SQLDB, linkservice(SQLDB), Table(choose tblorder1)), Mapping(Import Schemas)] --> debug, publish
```

16. Delete Activity :

```
# Delete Activity for Files, Folders only, Not for SQL Tables  
drag delete[Source(dataset:ADLS, orders.csv, linkservice(ADLS)), Enable Logging, select same ADLS for log files] --> debug, publish
```

17. Copy data with Stored Procedure :

```
Create SQL DB, Table with data, Create StoreProcedure as below  
Create Procedure nn_proc1(@Country varchar(20)) as  
Select ProductName, sum(SalesAmount) TotalSales from Orders where Country=@Country group by ProductName  
Exec nn_proc1 'Italy'  
drag Copy[Source(dataset:SQLDB, linkservice(SQLDB)), StoreProcedure(choose nn_proc1), ImportParameters(Country, string, Italy), Sink(dataset:ADLS, output, linkservice(ADLS))] --> debug, publish
```

18. Stored Procedure Activity :

```
drag StoreProcedure --> Settings(linkservice(SQLDB), StoredProcedure(choose nn_proc1), Parameters(Import)-->(Country, string, Italy))--> debug, publish  
Create Procedure nn_proc2 as  
truncate table SalesData  
insert into SalesData(ProductId, ProductName, InvoiceNum, Qty, Sales)  
Select P.ProductId, P.ProductName, S.InvoiceNum, S.Qty, S.Sales from Products P join SalesTran S on P.ProductId = S.ProductId  
drag StoreProcedure --> Settings(linkservice(SQLDB), StoredProcedure(choose nn_proc2)) --> debug, publish
```

19. Lookup Activity :

```
# Lookup Activity read & display file's inside data(Max 5000 rows/4 Mb) along with count of total rows  
drag Lookup --> Settings(dataset:ADLS, orders.csv, linkservice(ADLS), deselect 'first row only') --> debug, publish
```

20. Filter Activity :

```
ADF Pipeline --> Variable(numbers, Array, [10, 20, 30, 40, 50, 60])  
drag Filter-->Settings(Items(dynamic,@variables('numbers')), Condition(dynamic,@greater(item(), 30)))--> debug, publish # output: 40, 50, 60
```

21. If Activity :

```
ADF pipeline --> variable(x, int, 100), (Result, string)  
drag If --> Activities[Expression(dynamic, @equals(variables('x'), 100)), True(SetVariable1), False(SetVariable2)]  
SetVariable1(Settings(Name(result), Value(Yes))), SetVariable2(Settings(Name(result), Value(No)))
```

22. Switch Activity :

```
ADF pipeline --> variable(x, string, 1000), (Result, string)  
drag Switch --> Activities[Expression(dynamic, @variables('x')), AddCase(+), [1000-->Activities(SetVariable1, result, 1k)], [2000-->Activities(SetVariable2, result, 2k)]] --> debug, publish
```

23. Script Activity :

```
drag Script --> Settings(linkservice(SQLDB), Script[(Query), Alter table orders add qty int]) --> debug, publish
```

24. Validation Activity :

```
drag Validation --> Settings(dataset: ADLS, orders.csv) --> debug, publish # until timeout it will check continuously whether file exists or not
```

25. Convert csv --> Json :

```
drag copy --> Source(dataset:ADLS, orders.csv, linkservice(ADLS)), Sink(dataset:ADLS, Json, output, linkservice(ADLS)) --> debug, publish
```

26. Copy Json --> SQL DB Table :

```
Create ADLS with Employees.Json file and Create SQL DB with table(nn_emp) same schema of Json file  
drag copy --> Source(dataset:ADLS, Employees.json, linkservice(ADLS)), Sink(dataset:SQLDB, linkservice(SQLDB), choose dbo.nn_emp, Mapping(import schema)) --> debug, publish
```

27. Execute Pipeline Activity :

```
drag Executive Pipeline --> Settings(choose needed pipeline) --> debug, publish
```

28. Copy data if File Exists :

```
GetMetadata --> If --> [True(Copy) / False(Fail)]  
drag Get Metadata --> Settings(dataset:ADLS, orders.csv, linkservice(ADLS), FieldList(Exists))  
drag If --> [Expression(dynamic, @activity('File Exists').output.exists), True(drag Copy), False(drag Fail activity)] --> debug, publish  
Copy-->[Source(dataset:ADLS, linkservice(ADLS)), Sink(dataset:SQLDB, linkservice(SQLDB), Table(dbo.orders),
```

```
Pre-Copyscript(truncate table dbo.orders)))  
Fail --> Settings[Fail message(Orders file is Not available), ErrorCode(404)]
```

29. Parameters Vs Variables :

Parameter: we need to give its value at before execution of pipeline and it can't be changed during pipeline execution
Variable: This value can be update any no of times during execution also by using SetVariable activity

30. Delete Blank Files :

```
Get metadata1 --> Filter --> ForEach(GetMetadata2 --> If-->[True(delete) / False()])  
ADFPipeline --> Variable(FileName,string)  
drag Get Metadata1 --> Settings(dataset:ADLS,csv,input,linkservice(ADLS),FieldList(ChildItems))  
Filter --> Settings(Items(@activity('Get All Files').output.childItems,condition(@equals(item().type,'File'))))  
ForEach --> Settings(Items(@activity('Filter files').output.Value)),Activities [SetVariable(FileName,@item().name), GetMetadata2,  
    If(True(delete))] --> debug, publish  
Get Metadata2 --> Settings(dataset:ADLS,csv,input/@dataset().Sourcefilename,linkservice(ADLS),Parameter(Sourcefilename,string),  
    Sourcefilename(@variables('FileName'),FieldList(size)))  
If --> Settings(Expression(@lessOrEquals(activity('Get FileSize').output.size,1024)),True(delete activity),False())  
    drag delete--> [Source(dataset:ADLS,csv,input/@dataset().Sourcefilename,linkservice(ADLS),  
        Parameter(Sourcefilename,string), Sourcefilename(@variables('FileName'))),select same ADLS for logs]
```

31. Copy header less csv

```
# Copy Header Less csv  
Copy --> Source[dataset:ADLS,orders.csv, unchecked first row header], Sink[dataset:AzureSQLDB,Table(dbo.Orders),Mapping(import schema),  
align columns orderwise carefully]
```

32. Retry Logic

33. Copy Behaviour

34. Max Rows per File

```
Copy --> Source[dataset:ADLS,orders.csv], Sink[dataset:ADLS,output,file ext(.csv), Max rows per file(200)]
```

35. Split data single criteria

```
Copy --> Source[dataset:ADLS,orders.csv], Sink[dataset:ADLS,output,file ext(.csv), Max rows per file(200)]  
# split data single criteria  
keep ready Orders table in Azure SQLDB  
ADF pipeline --> variable(Country,string)  
Lookup --> Settings(dataset:AzureSQLDB,Query(Select distinct Country from orders))  
ForEach --> Settings(items(@activity('GetCountries').output.value), activity(SetVariable(Country,@item().COUNTRY) --> copy))  
Copy --> Source[dataset:AzureSQLDB,Query(@replace('Select OrderDate,ProductKey,EnglishProductName,SalesAmount,TaxAmt from orders where  
COUNTRY='countryname','countryname',variables('Country')))]  
Sink[dataset:ADLS,csv,output/@concat(dataset().CountryName,'.csv'),parameter(CountryName,string),CountryName(@variables('CountryName'))]
```

36. Split data multiple criteria

37. Consolidate data from Multiple Files

38. Consolidate data from Multiple Folders

39. Copy data custom Mappings (Single Table)

40. Copy data custom Mappings (Multiple Tables)

41. Copy data pipe character:

here create ordersNew table in Azure SQL DB with same schema of ADLS csv file and after copy in ADF, data will be copied into table
ADF --> pipeline --> Copy[Source(dataset:ADLS,csv,orderspipe.csv,Column delimiter(pipe(|))),
Sink(dataset:AzureSQLDB,Table(dbo.ordersNew))]

42. Copy data quote character:

here create ordersNew table in Azure SQL DB with same schema of ADLS csv file and after copy in ADF, data will be copied into table
ADF --> pipeline --> Copy[Source(dataset:ADLS,csv,ordersquote.csv, Quote Character(DoubleQuote(""))),
Sink(dataset:AzureSQLDB,Table(dbo.ordersNew))]
if we not select double quote option in copy, data inside of double quotes will be splitted and total table columns get disturbed

43. Introduction to dataflows:

44. Select Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug  
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)  
(+)--> Select(manually arrange/rename/delete columns)  
(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblSelect))--> debug,publish
```

45. Sort Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug  
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)  
(+)--> Sort[Sort Conditions(Country,Ascending),(SalesAmount,Descending),optimize(singlepartition)]  
(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblSort))--> debug,publish
```

46. Filter Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug  
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)  
(+)--> Filter(Filter on(Expression,Country=='India')-->Save&Finish,optimize(singlepartition))  
(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblFilter))--> debug,publish
```

47. Derived Column Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug  
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)  
(+)--> DerivedColumn(COLUMNS(Year,Expression(year(OrderDate)),Month,Expression(month(OrderDate)),Day,Expression(dayOfMonth(OrderDate)),  
    MonthName,Expression(toString(OrderDate,'MMM')),Tax,Expression(SalesAmount*0.05))-->Save&Finish,optimize(singlepartition))  
(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblderived))--> debug,publish
```

48. Conditional Split Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)
(+)--> Conditional Split(SplitCondition(India,Condition(Country=='India'),France,Condition (Country=='France'),Other) -->
    Save&Finish,optimize(singlepartition))
(+)-->Sink1(dataset:SQLDB,Newtable(dbo.tblSplitIndia))--> debug,publish
(+)-->Sink2(dataset:SQLDB,Newtable(dbo.tblSplitFrance))--> debug,publish # here Sinks adding parallel way with each split
```

49. Cast Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,orders.csv) # here we not doing datatype matching in projection, instead of this, we using Cast transform
(+)-->Cast(Columns choose & map needed datatype)--> optimize(singlepartition)
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblCast))--> debug,publish
```

50. Surrogate Key Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)
(+)-->SurrogateKey(Settings(keyColumn(id),startvalue(1),stepvalue(1))), optimize(singlepartition)
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblsgkey))--> debug,publish
```

51. Aggregate Transformation :

```
# case-1
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)
(+)-->Aggregate(groupBy(Country,EnglishProductName),Aggregates(TotalSales,Expression(round(sum(SalesAmount),0),
    TotalTax,Expression(sum(TaxAmt)))) --> Save&Finish optimize(singlepartition)
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblagg))--> debug,publish

# case-2
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)
(+)-->Aggregate(groupBy(Expression(SalesYear,year(OrderDate)),SalesMonth,month(OrderDate)))) --> Save&Finish optimize(singlepartition)
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblagg))--> debug,publish
```

52. Pivot Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,orders.csv),Projection(detecting datatype,set datatypes matching as per our need)
(+)-->pivot(Settings(Groupby(EnglishProductName),pivotkey(Country),pivotedColumns(lateral(sum(SalesAmount))))),
optimize(singlepartition)
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblpivot))--> debug,publish
```

53. Unpivot Transformation :

54. Rank Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug # Note: if check dense, it gives dense_rank wise
dataflow:Source(dataset:ADLS,Rank.csv),Projection(detecting datatype,set datatypes matching as per our need)
(+)-->Rank(Settings(RankColumn(Ranks),SortConditions(SalesAmount,Descending))),optimize(singlepartition)
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblpivot))--> debug,publish
```

55. Window Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,Rank.csv),Projection(detecting datatype,set datatypes matching as per our need)

# Case-1
(+)-->Aggregate(groupBy(OrderDate),Aggregates(TotalSales,Expression(round(sum(SalesAmount),2)))) --> Save&Finish
optimize(singlepartition)
(+)-->Window(Settings(Over(),Sort(OrderDate,Ascending),WindowColumns(CumSales,Expression(sum(TotalSales)))) --> Save&Finish,
optimize(singlepartition) # if check dense, it gives dense_rank wise
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblwindow1))--> debug,publish

# case-2
(+)-->Aggregate(groupBy(OrderDate),Aggregates(TotalSales,Expression(round(sum(SalesAmount),2)))) --> Save&Finish
optimize(singlepartition)
(+)--
>Window(Settings(Over(Expression(year(OrderDate),OrderYear),(month(OrderDate),OrderMonth)),Sort(OrderDate,Ascending),WindowColumns(CumS
ales,Expression(sum(TotalSales)))) --> Save&Finish, optimize(singlepartition) # if check dense, it gives dense_rank wise
(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblwindow2))--> debug,publish
Select * from tblwindow2 order by orderyear,ordermonth # run this in Azure SQLDB query Editor
```

56. Union Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source1(dataset:ADLS,Set1.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source2(dataset:ADLS,Set2.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source3(dataset:ADLS,Set3.csv),Projection(detecting datatype,set datatypes matching as per our need)
Source1(+)-->Union(Settings(Unionby(Name/position),Unionwith(Source2),(Source3))),optimize(singlepartition)
Union(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblunion))--> debug,publish
```

57. Lookup Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source1(dataset:ADLS,Sales.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source2(dataset:ADLS,products.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source3(dataset:ADLS,Customers.csv),Projection(detecting datatype,set datatypes matching as per our need)
Source1(+)-->Lookup1(Settings(primarystream(source1),Lookupstream(source2),LookupCondition(Left(ProductKey)==Right(ProductKey))),
optimize(singlepartition)
Lookup1(+)-->Lookup2(Settings(primarystream(Lookup1),Lookupstream(source3),LookupCondition(Left(CustomerKey)==Right(CustomerKey))),
optimize(singlepartition)
Lookup2(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblLookup),Mapping(uncheck Auto-Mapping and delete duplicate columns manually))
--> debug,publish
```

58. Join Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source1(dataset:ADLS,Sales.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source2(dataset:ADLS,products.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source3(dataset:ADLS,Customers.csv),Projection(detecting datatype,set datatypes matching as per our need)
Source1(+)-->Join1(Left(Sales),Right(Products),Type(Inner),JoinCondition(Left(ProductKey)==Right(ProductKey))),optimize(singlepartition)
Join1(+)-->Join2(Left(Join1),Right(source3),Type(Left),JoinCondition(Left(CustomerKey)==Right(CustomerKey))),optimize(singlepartition)
Join2(+)-->Sink(dataset:SQLDB,Newtable(dbo.tblJoin),Mapping(uncheck Auto-Mapping and delete duplicate columns manual))--> debug,publish
```

59. Exists Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source1(dataset:ADLS,Sales.csv),Projection(detecting datatype,set datatypes matching as per our need)
dataflow:Source2(dataset:ADLS,products.csv),Projection(detecting datatype,set datatypes matching as per our need)
Source1(+)-->Exists(Left(Sales),Right(Products),Type(Exists/doesnot Exists),ExistsCondition(Left(ProductKey)==Right(ProductKey))),
optimize(singlepartition)
Exists(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblExists),Mapping(uncheck Auto-Mapping and delete duplicate columns manual))
--> debug,publish
```

60. Flatten Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source1(dataset:ADLS,Orders.json), SourceOptions(JsonSettings(Array of documents))
Source(+)--> Flatten(unroll by(Items),InputColumns(Map columns as per need))
Flatten(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblFlatten))--> debug,publish
```

61. Parse Transformation :

```
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,Excel,OrderDetails.xlsx, select needed sheet), SourceOptions(JsonSettings(Array of documents))
Source(+)--> Parse1[DelimitedText,DelimitedSettings(ColumnDelimiter(|),Columns[OrdDet,Expression(OrderDetails),Expression Type(Item as string, Qty as integer, Price as integer)]] --> save & Finish
Parse1(+)--> Parse2[Json,JsonSettings(ArrayOfDocuments,Columns[BatchDet,Expression(BatchDetails),Expression Type(Batchid as integer, site as string)]] --> save & Finish
Parse2(+)--> Sink(dataset:SQLDB,Newtable(dbo.tblParse),Mapping(uncheck Auto-Mapping and delete duplicate columns(OrderDetails,BatchDetails) and select all above formed sub-columns manually))--> debug,publish
```

62. Stringify Transformation :

```
# Stringify (convert Nested Json Array to strings)
ADF pipeline --> drag dataflow activity --> Enable dataflow debug
dataflow:Source(dataset:ADLS,json,EmployeesNew.json), SourceOptions(JsonSettings(Array of documents))
Source(+)--> Stringify(Format(Json),Columns(Phones,Expression(select 'phone' column)))
Stringify(+)--> Sink(dataset:ADLS,csv,output, Mapping(uncheck Auto-Mapping and delete past nested Phone column and select above formed String phone column manually))--> debug,publish
```

63. Integration Runtime :

Its a Compute Infra capacity provided for Executing Tasks on Work Nodes by using Notebooks/SQL queries...etc
Types of IR: Azure, Self-Hosted, Azure-SSIS
If we perform data integration within Azure, we need to select Azure IR, its fully Managed by Azure
If we perform data integration with on-premises from Azure, we need to select self-hosted IR, its fully controlled by us
Before Azure introduce, for ETL purpose SSIS is used. So run that Tasks, we need to select Azure-SSIS, its separate software needs to be install

64. Install Self-Hosted IR :

when whole organization data is maintained at centralized server at specific physical space, its called on-premises
To migrate this on-premises data into cloud, one Gateway(GW) include compute resources is needed to connect, this called self-hosted IR
ADF --> Manage section --> IR --> New(+) --> Azure,Self-hosted --> self-hosted --> Name,description --> create --> Express/manual setup
Manualsetup(download MSI, copy keys) --> complete Install --> Finish --> paste key --> Register --> Launch Config Manager

65. Copy data from Local to ADLS :

```
ADF --> Copy --> Source[dataset:Filesystem,csv,ls_local(selfHostedIR,Host(C:\), Username(DESKTOP-RS540U0\Narian),password(XXX<laptop password>))] --> create --> browse orders.csv file from Laptop downloads folder
Sink[dataset:ADLS,csv,ls_adls(ADLS),output container(ADLS)] --> debug, publish
In powershell(as administrator)--> cd 'C:\ProgramFiles\MicrosoftIntegrationRuntime\5.0\Shared'
--> ./dmgcmd -disableLocalFolderPathValidation
```

66. Copy data from Local to Azure SQL DB :

```
ADF --> Copy --> Source[dataset:Filesystem,csv,ls_local(selfHostedIR,Host(C:\), Username(DESKTOP-RS540U0\Narian),password(XXX<laptop password>))] --> create --> browse orders.csv file from Laptop downloads folder
Sink[dataset:AzureSQL-DB,ls_sql(AzureSQLDB),AutoCreateTable,Table(dbo.OrderMigrate)] --> debug, publish
In powershell(as administrator)--> cd 'C:\ProgramFiles\MicrosoftIntegrationRuntime\5.0\Shared'
--> ./dmgcmd -disableLocalFolderPathValidation
```

67. Copy data Local SQL DB --> Azure SQL DB :

```
ADF --> Copy --> Source[dataset:SQLserver,ls_local_sql(selfHostedIR,ServerName(DESKTOP-RS540U0),dbName(AdventureWorksDW),Username(DESKTOP-RS540U0\Narian),password(XXX<laptop password>))] --> create --> select
dbo.DimProduct Table
Sink[dataset:AzureSQL-DB,ls_sql(AzureSQLDB),AutoCreateTable,Table(dbo.productMigrate)] --> debug, publish
In powershell(as administrator)--> cd 'C:\ProgramFiles\MicrosoftIntegrationRuntime\5.0\Shared'
--> ./dmgcmd -disableLocalFolderPathValidation
```

68. Copy data of Multiple Tables from Local SQL-DB --> Azure SQL-DB :

69. Using Azure Key Vault :

```
Azure Key Vault --> RG, Name(nn-kv), Region, Pricing Tier, permission model(Vault Access policy) --> Review + Create
Secrets(+)--> Name(sqlpassword), Secret Value(narian@123) --> Create
KeyVault(Access Policies) --> Create(+)--> select all permissions, Next, select our ADF(nnadf) Name --> create
Copy --> Source[dataset:ADLS,csv,orders.csv], Sink[dataset:AzureSQL-DB,ServerName,DBName,Username,
```

```
AzureKeyVault(KeyVault-Linkservice(Name(ls_kv),keyvaultName(nn-kv))),SecretName(sqlpassword)] --> Create  
AutoCreateTable,Table(dbo.orderstest) --> debug, publish
```

70. Schedule Trigger :

Create Any ADF Pipeline --> Add Trigger --> New --> Name,Type(Schedule,startdate,Recurrence(Every 5
Minutes/Hours/days/weeks/Months),enddate) --> save
check pipeline Job runs in Monitor Tab. ADF --> Manage --> Triggers --> select required Trigger and stop it if not needed

Azure Synapse(7):

1. Introduction :

here SQL Pools are provided for Enterprise datawarehouse features. ADLS & ADF automatically integrated/embedded in this synapse
here Notebooks also there for processing data by pyspark jobs and integrated with PowerBI and Cosmos-DB also

2. SQL pools :

dedicated SQL pool: it is known as SQL DWHouse
Serverless SQL pool: it is known as SQL on-demand and built-in and its free

3. Dedicated SQL pool :

It allows to config custom compute(collection of analytical resources CPU,Memory,i/o), its much costly and measured as DWU
It is built on MPP Engine. App/User fires T-SQL commands to Control Node(Master), in this distributed query engine optimizes these
queries for parallel processing
Next it passes these to compute Nodes where perform the query tasks in parallel and here each having DMS(data movement service) helps
to move data across worker nodes to get results
compute nodes range from 1 to 60 Max. in compute nodes storage & compute not tied together and are billed separately based on usage

4. Serverless SQL pool (Built-in) :

Its fully managed by Microsoft and its built-in, fast and cost for only queries, we can query directly on ADLS files without loading in
synapse. It is Auto-Scalable.

5. Distribution Types :

Hash distribution: High query performance while in Joins, aggregations on large tables using Hash Function Algorithm on specific key
column to assign rows to distributors
Round-Robin distribution: when using staging table during data loads, it delivers fast performance. It distributes data evenly across
the table sequentially, but No further optimizations
Joins take much delay in this.
Replicated distribution: for smaller tables it is much faster. here full copy of table is cached to each compute node

6. Provision Azure Synapse Analytics :

```
Create Synapse workspace[RG, Name, Region, AccountName(nnadls), FileSystemName(mycontainer),username,password] --> create  
Synapse Studio --> data[workspace(schema,tables),linked(ADLS)], develop(SQL-queries,Notebooks,dataflow), Integrate(ADF-Pipelines),  
Monitor, Manage(SQL pools, LinkedServices,Triggers,IRs)  
SQL-scripts,Notebooks all need to be connect with SQLPools(Built-in/dedicated)
```

7. Working with Serverless SQL pool :

SynapseStudio --> Data --> linked --> rightclick on ADLS files(csv,parquet,json...etc) --> run query in synapse SQL format --> publish

Azure Devops:

1. Introduction :

Its an end-to-end service offered task management, code version, CI-CD ...etc[Az-Boards, Az-Repos, Az-Pipelines,Az-TestPlans,Az-Artifacts]
Az-Boards: It is Task Tracker(Assign,verify,approve/reject,re-assign) of team members similar to Jira Tool
Az-Repos: It maintains several code versions at centralised place and flexible integrable with many tools
Az-TestPlans: These are for QA testing
Az-Artifacts: To manage all dependencies from single place. It enables publish and consume software package versions and used as input
for CD pipelines
Az-Pipelines: it is CI-CD to Build, Test and Deploy code across many services

2. Sign-up Azure Devops :

After sign-up complete, sign-in dev.azure.com --> Create Organization --> Create Projects --> Repo, pipeline

3. Creating Repo :

Create Repo with various Branches[Feature-1,2,3 Branches --> develop branch --> main branch --> ADFpublish branch] and upload needed
files and clone this wherever need

4. Integrate ADF with Azure Devops :

we need to careful while login Az-Devops should be as Default Directory A/c, not with microsoft A/c, then only we can connect ADF with
Az-Devops
ADF --> Top Left side --> Setup Code Repo --> Configure --> Az-Devops --> default EntraID --> continue --> select our Organization,
Project, Repo and branch --> Apply
Select feature branch in ADF and create pipelines and publish, Next same will be replicated in Az-Devops repo also in same branch, Next
raise pull request and merge developments into main branch